

Proyecto intermodular DAW

Bookshelf

IES Alan Turing



Desarrollo de aplicaciones web
Tutor: Fernando Francisco Toro Rueda
Lugar y fecha defensa: IES Alan Turing, 15 de junio de 2026

Autores:
Jose María Caracuel Merino
Francisco Manuel Toledo Pérez
Jesús Moreno Bernal

ÍNDICE

1. INTRODUCCIÓN

- 1.1 Descripción del proyecto
- 1.2 Justificación
- 1.3 Objetivo

2. IDEA PRINCIPAL

3. HERRAMIENTAS USADAS

- 3.1 Backend (Laravel)
- 3.2 Frontend (Angular)

4. Base de Datos

5. Backend – API REST

6. Panel de Administración (Filament)

7. Frontend – Angular

8. Servidor – AWS y Terraform

9. Despliegue continuo – GitHub Actions

10. Herramientas Utilizadas

11. Conclusiones

1. INTRODUCCIÓN

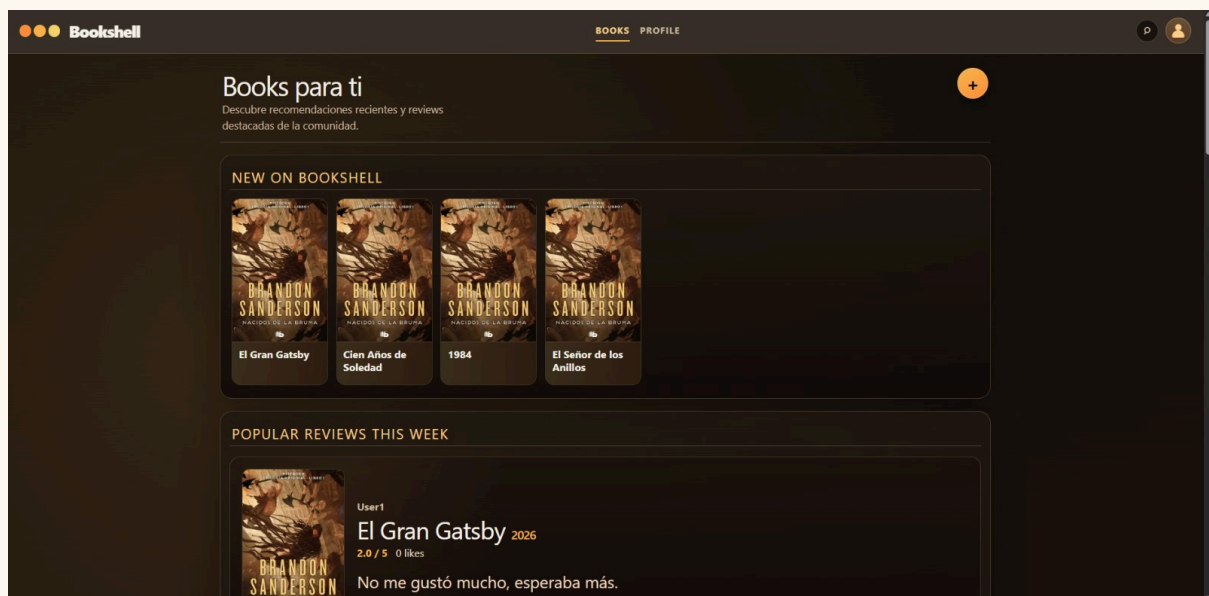
1.1 Descripción del proyecto

Bookshell es una aplicación web orientada a los amantes de la lectura. Permite registrarse como usuario, explorar un catálogo de libros, escribir reseñas y valoraciones, y gestionar el perfil personal. Cuenta además con un panel de administración para gestionar el contenido de la plataforma.

La aplicación está dividida en dos partes independientes que se comunican mediante una API REST:

Backend: desarrollado en PHP con Laravel 12, expone los endpoints de la API y aloja el panel de administración (Filament 3).

Frontend: desarrollado en Angular (TypeScript), consume la API y presenta la interfaz al usuario final.



1.2 Justificación del proyecto

El proyecto nace de la necesidad de tener una plataforma centralizada donde los lectores puedan descubrir libros, dejar sus opiniones y consultar las de otros usuarios, de forma similar a plataformas como Goodreads o StoryGraph, pero con un diseño propio y adaptado a las necesidades del proyecto académico.

Desde el punto de vista técnico, el proyecto permite aplicar de forma integrada todos los conocimientos adquiridos durante el ciclo: desarrollo web, bases de datos relacionales, APIs REST, despliegue en la nube y control de versiones.

1.3 Objetivo del proyecto

El objetivo principal es desarrollar una aplicación web full-stack completamente funcional que cumpla con los siguientes requisitos mínimos:

- Registro e inicio de sesión de usuarios con autenticación por token (Laravel Sanctum).
- Catálogo de libros con búsqueda y filtros.
- Sistema de reseñas y valoraciones por libro.
- Panel de administración para gestionar usuarios, libros y reseñas.
- Despliegue en un servidor cloud real (AWS EC2) mediante infraestructura como código (Terraform) y CI/CD (GitHub Actions).

2. IDEA PRINCIPAL

La idea central de Bookshell es crear una red social de lectura minimalista. Cualquier visitante puede consultar el catálogo y las reseñas, pero para escribir, editar o borrar contenido es necesario estar registrado.

El flujo principal del usuario es:

1. El usuario se registra o inicia sesión.
2. Explora el catálogo de libros.
3. Accede al detalle de un libro y lee las reseñas de otros usuarios.
4. Escribe su propia reseña con una valoración del 1 al 5.
5. Puede editar o eliminar sus reseñas desde su perfil.

El administrador dispone de un panel (Filament) accesible en /admin desde el que puede crear, editar y eliminar usuarios, libros y reseñas sin tocar la base de datos directamente.

3. HERRAMIENTAS USADAS

3.1 Para la parte de frontend:

HTML, CSS, ANGULAR, JAVASCRIPT Y TYPESCRIPT

Estructura del proyecto, estilos con librerías personales, funciones de métodos simples para hacer la página algo dinámica pero dejándole algunas funciones para integrar en el futuro REACT

```
frontend/
├── src/
│   ├── app/
│   │   ├── components/      # Componentes reutilizables
│   │   │   ├── book-card/
│   │   │   ├── review-card/
│   │   │   ├── navbar/
│   │   │   └── rating/
│   │   ├── pages/          # Páginas principales
│   │   │   ├── home/
│   │   │   ├── books/
│   │   │   ├── book-detail/
│   │   │   ├── profile/
│   │   │   └── auth/
│   │   ├── services/       # Servicios para consumir la API
│   │   │   ├── book.service.ts
│   │   │   ├── review.service.ts
│   │   │   └── auth.service.ts
│   │   ├── guards/         # Guards de rutas (autenticación y roles)
│   │   ├── interceptors/   # Interceptor HTTP para adjuntar el token
│   │   ├── models/         # Interfaces TypeScript
│   │   │   ├── book.model.ts
│   │   │   ├── review.model.ts
│   │   │   └── user.model.ts
│   ├── assets/
│   └── styles/
├── angular.json
├── package.json
└── tsconfig.json
```

3.2 Para la parte del backend:

PHP, LARAVEL, MYSQL Y PYTHON

Funciones para usar la base de datos, ya sea para guardar registros, actualizarlos, borrarlos o hacer peticiones para mostrarlos. Usamos una base de datos en mySQL en la que dejamos los datos de los usuarios, los libros de la página y las reseñas de estos, principalmente. Con Python, nos hemos conectado a una API incompleta para añadirla a la nuestra y así quitarnos la mayor parte de almacenar datos (todos los libros que tiene la API de google)

```
bookshell/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Api/          # Controladores de la API REST
│   │   │   │   ├── AuthController.php
│   │   │   │   ├── UserController.php
│   │   │   │   ├── LibroController.php
│   │   │   │   ├── ReviewController.php
│   │   │   │   └── User_ReviewController.php
│   │   ├── Models/          # Modelos Eloquent
│   │   │   ├── User.php
│   │   │   ├── Libro.php
│   │   │   └── Review.php
│   ├── Filament/
│   │   └── Admin/
│   │       └── Resources/    # Panel de administración
│   │           ├── UserResource.php
│   │           ├── LibroResource.php
│   │           └── ReviewResource.php
├── database/
│   ├── migrations/          # Esquemas de tablas
│   └── seeders/             # Datos de prueba
├── routes/
│   └── api.php              # Definición de endpoints
├── .env                    # Variables de entorno
└── terraform/              # Infraestructura AWS
```

4. Base de Datos

Tabla users

Campo	Tipo	Restricciones
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
name	VARCHAR(255)	UNIQUE, NOT NULL
roll	ENUM('admin', 'user')	DEFAULT 'user'
email	VARCHAR(255)	UNIQUE, NOT NULL
email_verified_at	TIMESTAMP	NULLABLE
phone	VARCHAR(255)	UNIQUE, NULLABLE
foto	VARCHAR(255)	DEFAULT 'default.png'
password	VARCHAR(255)	NULLABLE
localidad	VARCHAR(255)	NULLABLE
descripcion	TEXT	NULLABLE
remember_token	VARCHAR(255)	NULLABLE

Tabla libros

Campo	Tipo	Restricciones
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
titulo	VARCHAR(255)	NOT NULL
autor	VARCHAR(255)	NOT NULL
genero	ENUM('novela', 'fantasia', 'ciencia_ficcion', 'terror', 'misterio', 'romance', 'historia', 'biografia', 'poesia', 'ensayo', 'otro')	NOT NULL
portada	TEXT	NULLABLE
descripcion	VARCHAR(255)	NULLABLE
user_id	BIGINT UNSIGNED	FK → users.id

Tabla libros

Campo	Tipo	Restricciones
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT
user_id	BIGINT UNSIGNED	FK → users.id
libro_id	BIGINT UNSIGNED	FK → libros.id
comentario	TEXT	NOT NULL
valoracion	TINYINT	1–5

5. Backend – API REST

La API se sirve bajo el prefijo /api. Todos los endpoints que modifican datos requieren autenticación mediante Bearer Token (Laravel Sanctum).

Autenticación

Método	Endpoint	Descripción	Auth
POST	/api/register	Registro de nuevo usuario	No
POST	/api/login	Inicio de sesión, devuelve token	No
POST	/api/logout	Cierre de sesión, invalida token	Sí

Usuarios

Método	Endpoint	Descripción	Auth
GET	/api/usuarios	Listar todos los usuarios	No
GET	/api/usuarios/{id}	Ver usuario por ID	No
GET	/api/usuarios/buscar/{name}	Buscar usuario por nombre	No
PUT	/api/users/{id}	Actualizar datos del usuario	Sí
DELETE	/api/users/{id}	Eliminar usuario	Sí

Libros

Método	Endpoint	Descripción	Auth
GET	/api/libros	Listar todos los libros	No
GET	/api/libros/{id}	Ver libro por ID	No
GET	/api/libros/buscar/{title}	Buscar libro por título	No
GET	/api/libros/{libroId}/opiniones	Ver opiniones de un libro	No
POST	/api/libros	Crear libro	Sí
PUT	/api/libros/{id}	Actualizar libro	Sí
DELETE	/api/libros/{id}	Eliminar libro	Sí

Reseñas

Método	Endpoint	Descripción	Auth
GET	/api/reviews	Listar todas las reseñas	No
GET	/api/reviews/{id}	Ver reseña por ID	No
POST	/api/reviews	Crear reseña	Sí
PUT	/api/reviews/{id}	Actualizar reseña	Sí
DELETE	/api/reviews/{id}	Eliminar reseña	Sí

Relación Usuario-Reseña

Método	Endpoint	Descripción	Auth
GET	/api/user_reviews	Listar relaciones	No
GET	/api/user_reviews/{id}	Ver relación por ID	No
POST	/api/user_reviews	Crear relación	Sí
PUT	/api/user_reviews/{id}	Actualizar relación	Sí
DELETE	/api/user_reviews/{id}	Eliminar relación	Sí

```

1  {
2    "data": [
3      {
4        "id": 1,
5        "titulo": "El Gran Gatsby",
6        "autor": "F. Scott Fitzgerald",
7        "genero": "novela",
8        "descripcion": "Una historia de amor y tragedia en la era del jazz.",
9        "reviews": [
10         {
11           "id": 1,
12           "user": "Admin",
13           "rating": null,
14           "comment": null
15         },
16         {
17           "id": 2,
18           "user": "User1",
19           "rating": null,
20           "comment": null
21         },
22         {
23           "id": 9,
24           "user": "User2",
25           "rating": null,
26           "comment": null
27         }
28       ]
29     }
30   ]
31 }

```

6. Panel de Administración (Filament)

El panel de administración está construido con Filament 3 y es accesible en la ruta /admin. Solo los usuarios con roll = 'admin' pueden acceder.

Relación Usuario-Reseña

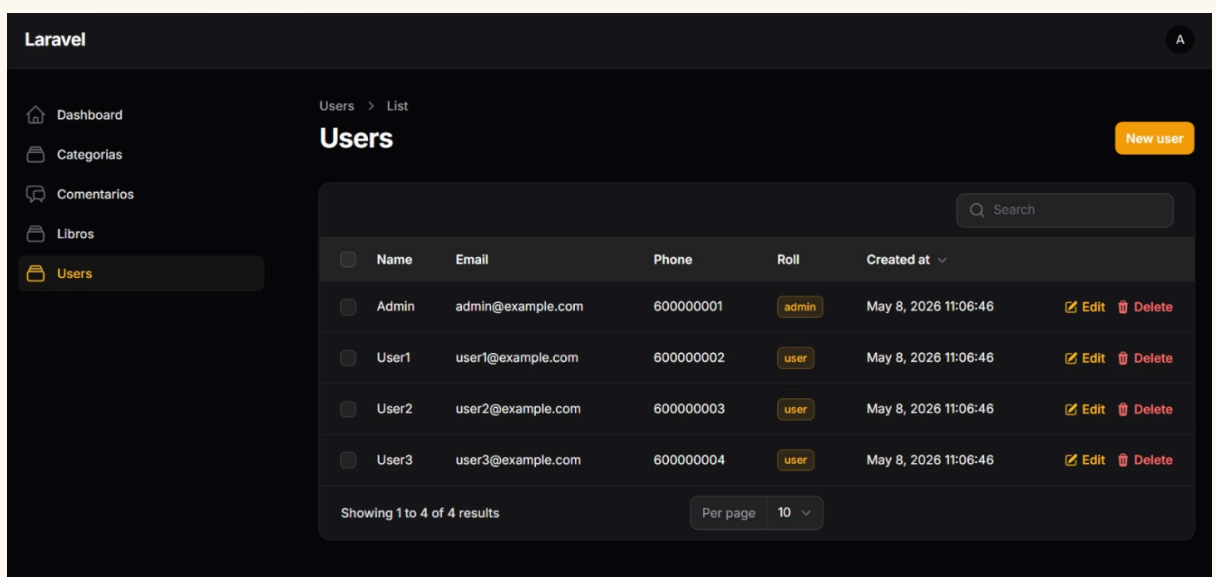
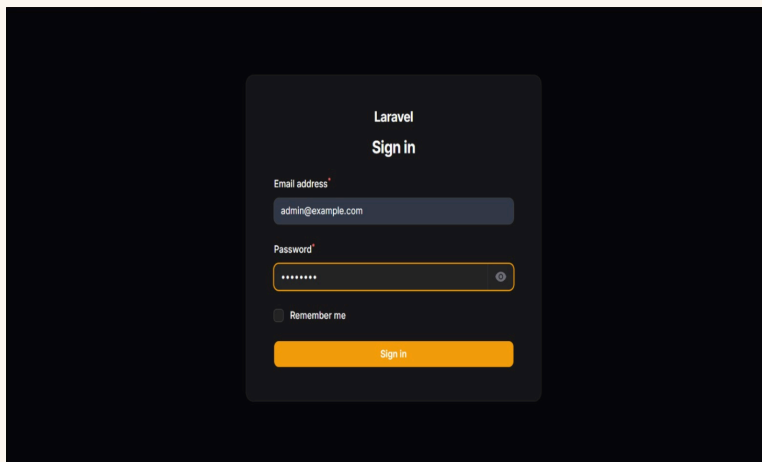
Recurso	Tabla	Operaciones CRUD
Usuarios	users	Crear, Ver, Editar, Eliminar

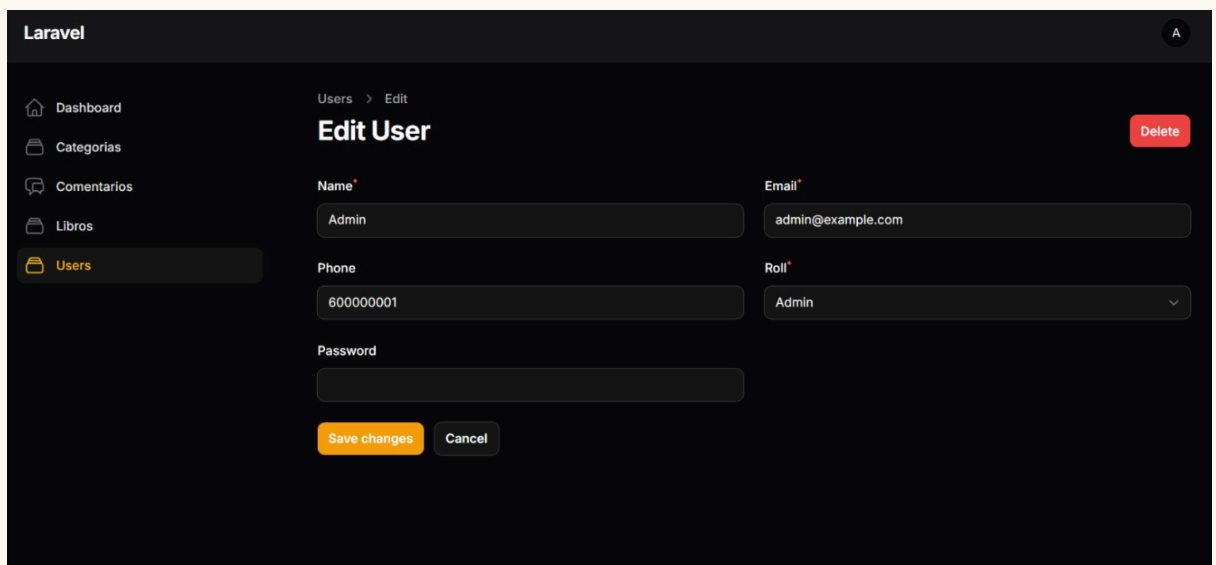
Libros	libros	Crear, Ver, Editar, Eliminar
Reseñas	reviews	Crear, Ver, Editar, Eliminar

Control de acceso

El modelo User implementa la interfaz FilamentUser, que exige el método canAccessPanel. Solo los usuarios administradores pueden entrar:

```
public function canAccessPanel(Panels $panels): bool
{
    return $this->roll === 'admin';
}
```





8. Servidor – AWS y Terraform

La infraestructura del servidor está definida como código con Terraform y se despliega en AWS

Recursos AWS aprovisionados

Recurso	Descripción
EC2 (t2.micro)	Instancia con Ubuntu Server para alojar la aplicación
Security Group	Reglas de entrada: SSH (22), HTTP (80), HTTPS (443)
IP elástica	IP pública fija asociada a la instancia EC2

Configuración del servidor

El servidor usa Apache como servidor web con los siguientes módulos habilitados:

- mod_rewrite – para las rutas de Laravel y Angular.
- mod_proxy / mod_proxy_http – para redirigir peticiones /api al backend Laravel.

- mod_ssl – para HTTPS.

El backend Laravel sirve desde /var/www/html/bookshell y el frontend Angular compilado desde /var/www/html/bookshell-frontend.

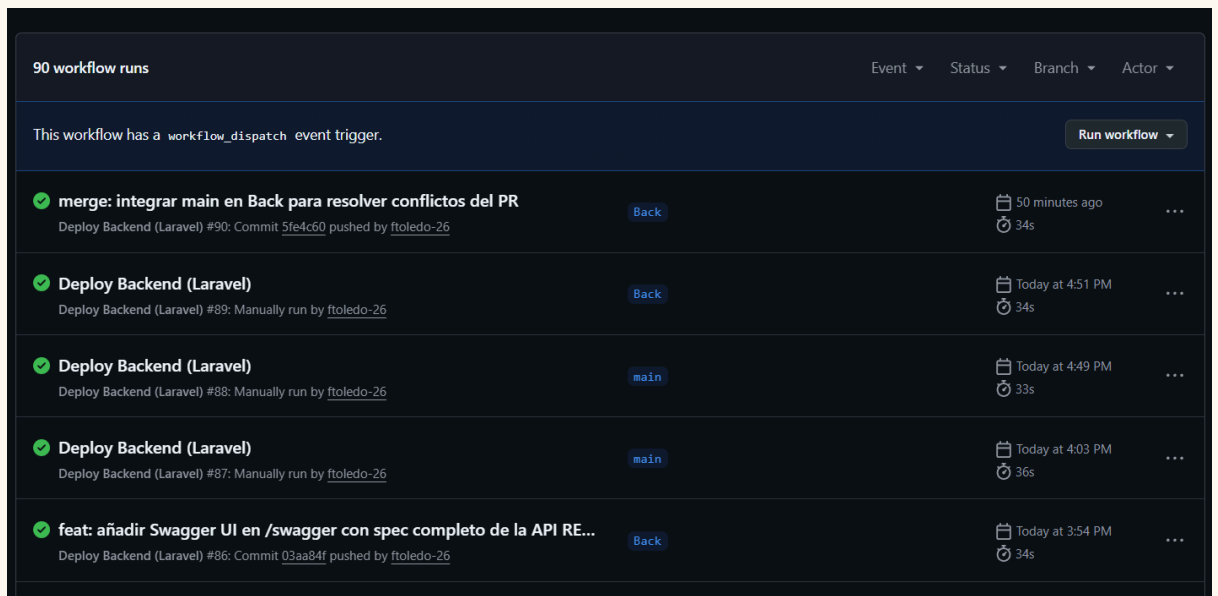
```
data.aws_ami.ubuntu: Reading...
data.aws_vpc.default: Reading...
data.aws_ami.ubuntu: Read complete after 0s [id=ami-00403f401ee6a4b98]
data.aws_vpc.default: Read complete after 0s [id=vpc-0fdedfcc3e4c1d6aa]
data.aws_subnets.default: Reading...
aws_security_group.bastion_sg: Refreshing state... [id=sg-00eae2bd90c262804]
aws_security_group.alb_sg: Refreshing state... [id=sg-028d9f49791b7db7f]
aws_lb_target_group.frontend: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:855016220711:targetgroup/bookshell-tg/951d3659993fc4b7]
aws_instance.bastion: Refreshing state... [id=i-0154ab2669e21530d]
aws_security_group.frontend_sg: Refreshing state... [id=sg-0f74a5b2c1f93b40f]
data.aws_subnets.default: Read complete after 1s [id=us-east-1]
aws_db_subnet_group.default: Refreshing state... [id=bookshell-db-subnets]
aws_lb.frontend: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:855016220711:loadbalancer/app/bookshell-alb/a6f81a1385c8ce61]
aws_security_group.backend_sg: Refreshing state... [id=sg-0e2255db7b34a5fc5]
aws_security_group.rds_sg: Refreshing state... [id=sg-03f7064063e44c377]
aws_lb_listener.http: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:855016220711:listener/app/bookshell-alb/a6f81a1385c8ce61/a04cc1746b309b98]
aws_db_instance.mysql: Refreshing state... [id=db-67G4UPX2LIOPNY5OX35XVZIJGQ]
aws_instance.backend: Refreshing state... [id=i-019008d61fdbdc255]
aws_eip.bastion_eip: Refreshing state... [id=eipalloc-0fe16f3579bf765e8]
aws_instance.frontend: Refreshing state... [id=i-0b4f7dde28be14e68]
aws_lb_target_group_attachment.frontend: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:855016220711:targetgroup/bookshell-tg/951d3659993fc4b7,i-0b4f7dde28be14e68,80]
aws_eip.frontend_eip: Refreshing state... [id=eipalloc-084cb008b1cf0e957]
```

9. Despliegue continuo – GitHub Actions

Se han configurado GitHub Actions para automatizar el despliegue en cada push a la rama principal.

Flujo del workflow

1. Se hace push a la rama main.
2. GitHub Actions conecta al servidor EC2 por SSH.
3. Ejecuta git pull en el servidor para obtener el código actualizado.
4. Ejecuta composer install --no-dev para instalar dependencias PHP.
5. Ejecuta php artisan migrate --force para aplicar migraciones pendientes.
6. Reinicia el servidor Apache.



10. Herramientas Utilizadas

Backend

Herramienta	Versión	Uso
PHP	8.2+	Lenguaje de programación del servidor
Laravel	12	Framework PHP para la API REST
Laravel Sanctum	4+	Autenticación por token para la API
Filament	3	Panel de administración CRUD
MySQL	8+	Base de datos relacional
Composer	2+	Base de datos relacional

Frontend

Herramienta	Versión	Uso
Angular	17+	Framework SPA para el frontend

TypeScript	5+	Tipado estático sobre JavaScript
HTML / CSS		Maquetación y estilos de la interfaz
npm	10+	Gestión de dependencias JavaScript

Infraestructura y DevOps

Herramienta	Uso
AWS EC2	Servidor de producción
Terraform	Infraestructura como código (IaC)
GitHub Actions	CI/CD: despliegue automático en cada push
Apache	Servidor web en producción
Git / GitHub	Control de versiones y repositorio remoto

Herramientas de desarrollo

Herramienta	Uso
VS Code	Editor de código principal
XAMPP	Entorno local PHP + MySQL para desarrollo
Postman	Prueba manual de los endpoints de la API
phpMyAdmin	Gestión visual de la base de datos en local

11. Conclusiones

El desarrollo de Bookshell ha permitido aplicar de manera práctica e integrada los conocimientos adquiridos a lo largo del ciclo formativo: desde el diseño de una base de datos relacional, pasando por la implementación de una API REST con autenticación, hasta el despliegue en infraestructura cloud real con automatización del proceso.

Entre los aspectos más destacables:

- La separación entre backend (Laravel) y frontend (Angular) mediante una API REST ha resultado en una arquitectura limpia, escalable y fácilmente mantenible.
- El uso de Filament ha simplificado enormemente la creación del panel de administración, evitando tener que construir interfaces CRUD desde cero.
- La integración de Terraform y GitHub Actions ha introducido buenas prácticas de DevOps en el proyecto, haciendo que el despliegue sea reproducible y automatizado.

Como posibles mejoras futuras se plantean:

- Añadir sistema de "libros favoritos" y listas de lectura personalizadas por usuario.
- Implementar paginación real en el backend para el catálogo de libros.
- Añadir notificaciones en tiempo real (Laravel Echo + Pusher) cuando alguien comenta un libro.
- Ampliar la cobertura de tests unitarios y end-to-end.



Recursos adicionales

Vídeo de demostración (YouTube):

<https://youtu.be/NuBrAkYFazA?si=-IFGRMvcEkk9d-mY>

Plan de empresa (Canva)

<https://canva.link/6knr3oj449jrly7>

Planificación del trabajo (Trello)

<https://trello.com/invite/b/6a09deb05b2468e0e2a4d076/ATTI03625ced21b90bdd1c5302764cdbf0c279B93C89/bookshelf>

Repositorio de trabajo (GitHub)

<https://trello.com/invite/b/6a09deb05b2468e0e2a4d076/ATTI03625ced21b90bdd1c5302764cdbf0c279B93C89/bookshelf>